TECHZEPHYR 4.0 · IIT BHUBANESWAR · SKETCH2SHAPE AI

Sketch2Shape Architecture Decision

An evaluation of three ways to orchestrate the sketch-to-3D pipeline for a 24-hour build, judged against the problem statement and its 100-point rubric. Written 31 October 2026, before the event. No implementation code.

Claims about models and libraries carry a tag. `CHECKED` means confirmed against an official repository, model card or docs page on 31 Oct 2026 (sources at the end). `TEST` means an estimate or assumption that the first spike has to measure.

Decision in brief

`BUILD`	A deterministic Python pipeline driven by a small explicit state machine. It has two bounded quality gates (sketch fidelity after image generation, mesh validity after reconstruction), a fallback ladder sized to the GPU, and a per-run folder that keeps every intermediate.
`SKIP`	LangGraph as the orchestrator, and any LLM agent in the control loop. Neither addresses where this problem is hard.
`BORROW`	Conditional routing from Approach 2, as a transition table. The evaluator from Approach 3, as deterministic scorers. One optional call to a vision-capable LLM, using the team's API key, names the sketch, with CLIP as the fallback.
`SPEND HOURS ON`	The sketch-to-image domain gap, best-of-N candidate ranking, mesh repair and export conventions, and a clean-machine install test. Output quality, pipeline reliability and reproducibility together carry 60 of the 100 points.
`SETTLE FIRST`	Compute is decided: free only, on Kaggle notebooks with two 16 GB T4 GPUs. Still open: what the judges' "evaluation environment" is, and whether setup and weight downloads may happen before the clock starts.
`PACE`	Submit as soon as the definition of done in section 7 is met, targeting about 10–12 hours after the start. Nothing is added to fill the rest of the 24 hours.

1. Understanding the problem statement
2. Approach 1: conventional pipeline
3. Approach 2: LangGraph workflow
4. Approach 3: agentic LangGraph
5. Other architectures
6. Side-by-side comparison
7. 24-hour estimates
8. Final recommendation
9. Architecture diagram
10. Risks and decisions to settle

01 Understanding the problem statement

1.1 What the PS requires, and how firmly

The PS mixes hard requirements with suggestions. "Must" rows are the ones a judge can fail you on. Everything in later sections that is not in this table is our own recommendation.

Requirement	Status	PS section
In-app drawing canvas with brush, eraser and reset	Must*	2A

Requirement	Status	PS section
Upload of a sketch image in a common format	Must*	2A
Works with no text prompt. The prompt only refines type, material, colour or style	Must	2A
Preprocessing (resize, background, contrast, noise) where needed	Should	2A
Output stays meaningfully related to the sketch, not produced from the prompt alone	Must	2B
Real 3D geometry with depth and volume. A rendered image, screenshot or flat textured plane fails	Must	2C, 2D
Export as `.glb` or `.obj`	Must	2D, Scope
Colours, materials or textures where the method supports them	Should	2C
Basic mesh validation: the file opens, geometry and textures survive export	Should	2D
Interactive 3D preview and download in the app	Encouraged	2D
Disclose every pretrained model, external API and third-party component	Must	Pipeline
README covering objective, architecture, dependencies, install, run, model choices, inputs and outputs, **time to generate a model**, **hardware used**, limitations	Must	Submission
At least one generated `.glb` / `.obj`, plus example inputs and outputs	Must	Submission
Reproducible by following the README "in the available evaluation environment", including weight access and external services	Must	Judging, Submission
Runs live on a fresh sketch or test case chosen by the judges	Must (implied)	Judging
Chat or conversational interface	Not required	2A
Print-ready, engineering-grade or animation-ready topology	Out of scope	Scope

* The Scope section allows "either or both" input modes, while section 2A and the submission list ask for both. Build both. In Gradio the canvas and the upload widget together take under an hour.

1.2 Where the difficulty actually is

- **Domain gap.** Image-to-3D models are trained on rendered or photographed objects with shading. A line drawing has no shading, material or depth cues, so a raw sketch tends to reconstruct as a flat or noisy shape `TEST`. Stage 2 (sketch to shaded image) translates the sketch into the reconstructor's input domain. It is where the sketch's identity is kept or lost.
- **Fidelity against plausibility.** Strong sketch conditioning copies the lines but can give flat, drawn-looking images that reconstruct badly. Weak conditioning gives attractive objects that no longer match the sketch. This one trade-off, controlled mostly by the ControlNet conditioning scale, is the main tuning parameter of the whole system, and it cannot be tuned without a number that measures fidelity.
- **A reconstruction-friendly image.** Reconstruction works best on one centred object, fully in frame, on a plain background, with soft even light and no cast shadow or ground plane `TEST`. The prompt template, negative prompt and preparation stage exist to produce that image.
- **Reconstruction fails quietly.** A returned mesh is not necessarily a good one. Floaters, holes, lost thin parts (chair legs, handles), a flat back, wrong scale or orientation, baked-in lighting and texture seams raise no exception.
- **One GPU, two large generative models.** A diffusion model with ControlNet and a 3D shape-and-texture model compete for VRAM. Hunyuan3D-2 alone needs 6 GB for shape and 16 GB for shape plus texture `CHECKED`. Load order and GPU residency are an architectural concern.
- **Unknown judge environment.** Reproducibility (15 points) is assessed "in the available evaluation environment", which the PS does not describe. Compiled CUDA extensions, gated weights and tens of gigabytes of downloads make this the largest risk outside the models themselves.

None of these is an orchestration problem. The control flow is short and known before a run starts. The uncertainty sits inside the model calls, and it is handled by measuring outputs, not by reasoning about what to run next. Every comparison below follows from this.

1.3 Where the 100 points come from

Criterion	Pts	What earns the points	Responsible part of the architecture
Technical implementation	25	Model integration, code structure, preprocessing, error handling, sound decisions	Typed stage contracts, model manager, failure table (2.4), choices justified in the README
3D output quality and design	25	Recognisable, faithful to sketch and prompt, plausible geometry, texture quality	Model choice, stage-2 tuning, best-of-N scoring, mesh repair and normalisation
End-to-end validity and efficiency	20	Drawing to export always completes; runtime and resource use	Fallback ladder that never returns nothing, time budget, quality presets, measured timings
Code reproducibility	15	Judges install and run from the README with no hidden steps	Pinned environment, weight download script, Linux or Docker path, CLI smoke test
Documentation and README	15	Architecture, setup, attribution, samples, limitations, troubleshooting	Diagram, results table generated by a batch script, ablation with and without stage 2

The orchestration framework appears in none of these rows directly. It affects the score only through reliability, debuggability, and the hours it takes away from the rest.

---

02 Approach 1: conventional pipeline

A fixed sequence of Python functions, called in order by one controller. This section designs it concretely, because the stage design, contracts and failure table here are shared by every other approach.

2.1 Repository structure

```
sketch2shape/
  app.py              Gradio UI: canvas, upload, prompt, 3D viewer, downloads
  cli.py              same run, headless:  python -m sketch2shape.cli in.png --prompt "..."
  config.yaml         model ids + pinned revisions, steps, thresholds, budgets, GPU tier
  pipeline.py         the controller: calls stages, owns every retry and fallback decision
  contracts.py        small dataclasses passed between stages (paths + metadata)
  models.py           ModelManager: lazy loading, GPU residency, CPU offload
  runstore.py         runs/<run_id>/ folders, content-hash cache, manifest.json
  stages/
    preprocess.py     OpenCV clean-up
    understand.py     LLM sketch naming (optional) or CLIP label + prompt builder
    generate.py       SDXL + scribble ControlNet, N candidates
    score.py          fidelity metrics
    prepare.py        background removal, crop, recentre
    reconstruct.py    adapters: hunyuan3d2 (primary), shape + projected colour (fallback)
    validate.py       trimesh checks + repair
    export.py         GLB + OBJ zip, Y-up, scale, thumbnails, reload test
scripts/
  download_weights.py pinned Hugging Face revisions, prints total size
  eval_batch.py       runs the test sketches, writes the README results table
examples/             input sketches + generated outputs for the submission
tests/                smoke test on three sketches
```

2.2 Stages and hand-off contracts

Stage	What it does	Output contract	Libraries
1 Preprocess	Flatten canvas RGBA onto white. For photos of paper: even out lighting, raise contrast (CLAHE), auto-invert dark backgrounds, adaptive threshold, drop specks, close small stroke gaps. Crop to strokes with ~12% padding, square to 1024 px. Reject an empty canvas.	`SketchAsset`: line-art PNG, control image in the polarity the ControlNet expects `TEST`, filled silhouette mask, stroke statistics	OpenCV, NumPy
2 Understand	When an API key is set, one call to a vision-capable LLM names the object. Without a key, or on any API error, a CLIP zero-shot label over a fixed vocabulary (Quick, Draw! 345 classes plus common products and furniture) with a confidence. Merge with the user's prompt. Build a positive prompt from a reconstruction-friendly template and a negative prompt (multiple objects, cropped, text, shadow, ground, scene, line art).	`Intent`: label, confidence, prompt, negative prompt	LLM provider SDK (optional), open_clip
3 Generate	N candidates (default 4) in one batch from a fixed seed list, 1024 px.	`Candidates`: image paths, seeds, parameters	diffusers: SDXL base + xinsir scribble ControlNet
4 Score	Per candidate: stroke recall and precision against the candidate's edges (distance-transform tolerance), silhouette IoU against the sketch mask, CLIP prompt similarity, and agreement between the candidate's CLIP label and the sketch label. Weighted composite.	`Ranked`: candidates with per-metric scores	OpenCV, open_clip, rembg
5 Prepare	Background removal, with a white-threshold matte as fallback (the prompt asked for a white background). Crop, recentre, add margin, resize to the reconstructor's input size.	`PreparedImage`: RGBA path, mask	rembg (BiRefNet or ISNet)
6 Reconstruct	Shape, then texture, behind one adapter interface so models can be swapped or stepped down.	`RawMesh`: path, model id, parameters	Hunyuan3D-2 (primary)
7 Validate	Checks listed below, plus one repair pass.	`MeshReport`: pass/fail per check, repaired mesh path	trimesh
8 Export	Centre, scale to unit size, Y-up as glTF expects. GLB with embedded texture, OBJ + MTL + PNG zip, reload test of the written file.	`ExportBundle` + `manifest.json`	trimesh

2.3 Data flow and intermediate results

- **Stages are plain functions.** Each takes the run context and the previous contract and returns the next contract. Contracts hold file paths and metadata. Images and meshes live on disk; loaded models live in the ModelManager. No stage calls another stage or decides what runs next.
- **One folder per run.** `runs/<run_id>/` holds numbered intermediates (`01_lineart.png` through `08_model.glb`) and a `manifest.json` with inputs, seeds, model revisions, parameters, scores, timings and warnings. That folder is the debugger, the source of the README results table, and the reproducibility record.
- **Content-hash cache.** Each stage's output is keyed by a hash of its input and parameters, so re-running only reconstruction while tuning takes seconds instead of a full run. During a 24-hour build this is worth more than any orchestration feature.
- **GPU residency.** The ModelManager loads each model once and keeps only what fits. On Kaggle's two T4s it keeps SDXL on GPU 0 and Hunyuan3D-2 on GPU 1; on a single 16 GB card it moves the diffusion pipeline to CPU before loading the 3D model.

2.4 Failures, invalid outputs and retries

This table is the real error-handling design. It is the same under all three approaches; only the place where it is written changes.

Failure	How it is detected	Response	Cap
Empty or unreadable sketch	Stroke-pixel ratio below a floor	Stop with a message asking the user to redraw	0
Candidates unfaithful to the sketch	Best composite score below threshold	Regenerate with new seeds and conditioning scale +0.1	2 rounds

Failure	How it is detected	Response	Cap
Background removal empty or over-cut	Mask area under 5% or over 95% of the frame	Switch method (BiRefNet ↔ white threshold)	1
CUDA out of memory	Exception	Free cache, offload other models, retry at lower resolution or fewer candidates	1
Mesh empty, flat or exploded	Validation checks	New seed, then next rung of the fallback ladder	1 + ladder
Texture missing or uniform	Texture image absent or near-zero variance	Retry texturing once, else colour projected from the prepared image	1
Export does not reload	Reload of the written file fails	Re-export through the other format path	1
Time budget spent	Wall clock checked at every transition	Stop loops, export the best result so far with a warning	n/a

Mesh validation checks: the exported file reloads; vertex and face counts are above a minimum; every coordinate is finite; the shape is not flat (smallest bounding extent at least ~8% of the largest `TEST`), which directly covers the PS's "flat plane" failure; fragments under ~2% of surface area are removed; normals and winding are fixed; faces are reduced to ~100–200k for a responsive viewer; UVs and a non-uniform texture are present. A front-silhouette IoU against the prepared mask is added only if the spike confirms each model's output orientation. Watertightness is reported, not required.

2.5 Are plain functions and conditionals enough?

Yes. The control flow is a sequence, two bounded loops and a fallback ladder, which comes to roughly 150–250 lines in one controller module. A framework would not remove any of the work in 2.4. The real risk with plain Python is retry logic leaking into the stages as nested `if` blocks; keeping every control decision in `pipeline.py` prevents it.

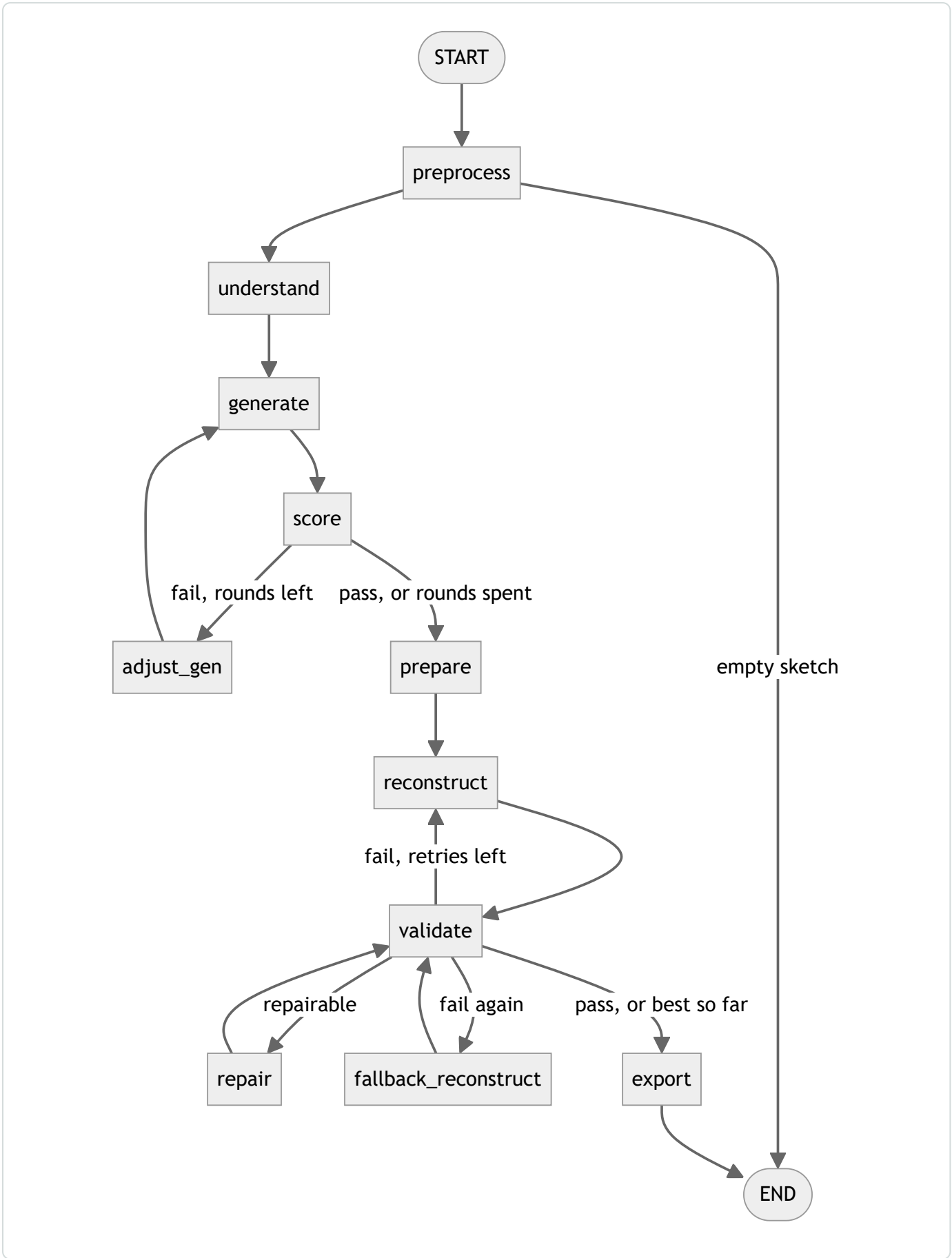
VERDICT As a strictly linear chain with no gates, this approach exports whatever the models return. It nearly always produces a file, sometimes a bad one, with no signal either way. With the gates from 2.4 it becomes the recommended design (section 5).

---

03 Approach 2: LangGraph workflow

The same pipeline as a `StateGraph`. There is no LLM in it, so it is a deterministic workflow with conditional routing, not an agent.

3.1 The graph



Edges with labels are conditional; the others are fixed. Every node wraps the same stage function as Approach 1.

3.2 Shared state

Field	Holds	Note
`run_id`, `sketch_path`, `prompt`	Inputs	Prompt may be empty
`lineart_path`, `control_path`, `mask_path`	Preprocess output	Paths, never arrays
`label`, `label_conf`, `prompt_final`, `negative_prompt`	Understand output	
`candidates`	List of {path, seed, scores}	Written by generate, scored by score
`selected_path`, `prepared_path`	Chosen and prepared image	
`mesh_path`, `mesh_report`, `exports`	3D results	
`params`	Conditioning scale, seed offset, reconstruction rung	Changed only by adjust and router logic
`attempts`	Counters for generate, reconstruct, repair	Merge reducer
`warnings`, `errors`	Messages for the UI and manifest	Append reducer
`timings`	Seconds per node	Merge reducer

State holds only JSON-serialisable values so a checkpointer can persist it. Loaded GPU models stay in a module-level ModelManager outside the graph.

3.3 Edges

- **Fixed:** START → preprocess; understand → generate → score; adjust_gen → generate; prepare → reconstruct → validate; repair → validate; fallback_reconstruct → validate; export → END.
- **After preprocess:** an empty sketch ends the run with a message for the user.
- **After score** (`route_after_score`): pass goes to prepare; a fail with rounds left goes to adjust_gen; a fail with none left goes to prepare with the best candidate and a warning.
- **After validate** (`route_after_validate`): pass goes to export; repairable goes to repair once; a fail with retries left goes back to reconstruct with a new seed; a second fail goes to fallback_reconstruct; nothing usable exports the best mesh so far, flagged, or ends with a clear error.

3.4 Failures and retries in LangGraph terms

- **Exceptions:** a `RetryPolicy` on `add_node` retries chosen exception types with backoff `CHECKED`. Use it for download and I/O hiccups. CUDA out-of-memory is not transient at the same settings, so the node handles it itself by lowering settings.
- **Quality loops:** conditional edges plus the attempt counters in state. The router functions are where the retry policy lives.
- **Backstops:** set `recursion_limit` explicitly on every invoke. LangGraph 1.2 and later also offer a per-node `timeout` and graph-wide defaults through `set_node_defaults` `CHECKED`. A timeout cannot stop a CUDA kernel already running in the same process, so it limits waiting, not GPU work.
- **Checkpointing:** with an in-memory or SQLite checkpointer keyed by `run_id`, a crashed run resumes from its last completed node.
- **Streaming:** `graph.stream(..., stream_mode="updates")` feeds node results to the Gradio progress panel.

3.5 What LangGraph adds over plain Python, and what it costs

ADDS

- Routing declared as a graph, and a Mermaid diagram generated from the code for the README.
- Checkpoint and resume of a run.
- A standard streaming interface for progress.
- RetryPolicy and node caching built in.
- Node timings in LangSmith or Langfuse if wanted.
- A ready home for LLM features later, such as chat-based refinement.

COSTS

- State must stay serialisable; one PIL image in state breaks checkpointing.
- GPU stack traces pass through the runtime, an extra layer to read when debugging.
- Bridging `graph.stream` into Gradio's generator and queue needs care.
- One more pinned dependency for judges to install.
- About 1.5–3 hours more than plain Python, mostly integration and debugging.
- Judges may ask why an LLM-free, mostly linear flow uses an agent framework. The answer has to be one of the features on the left.

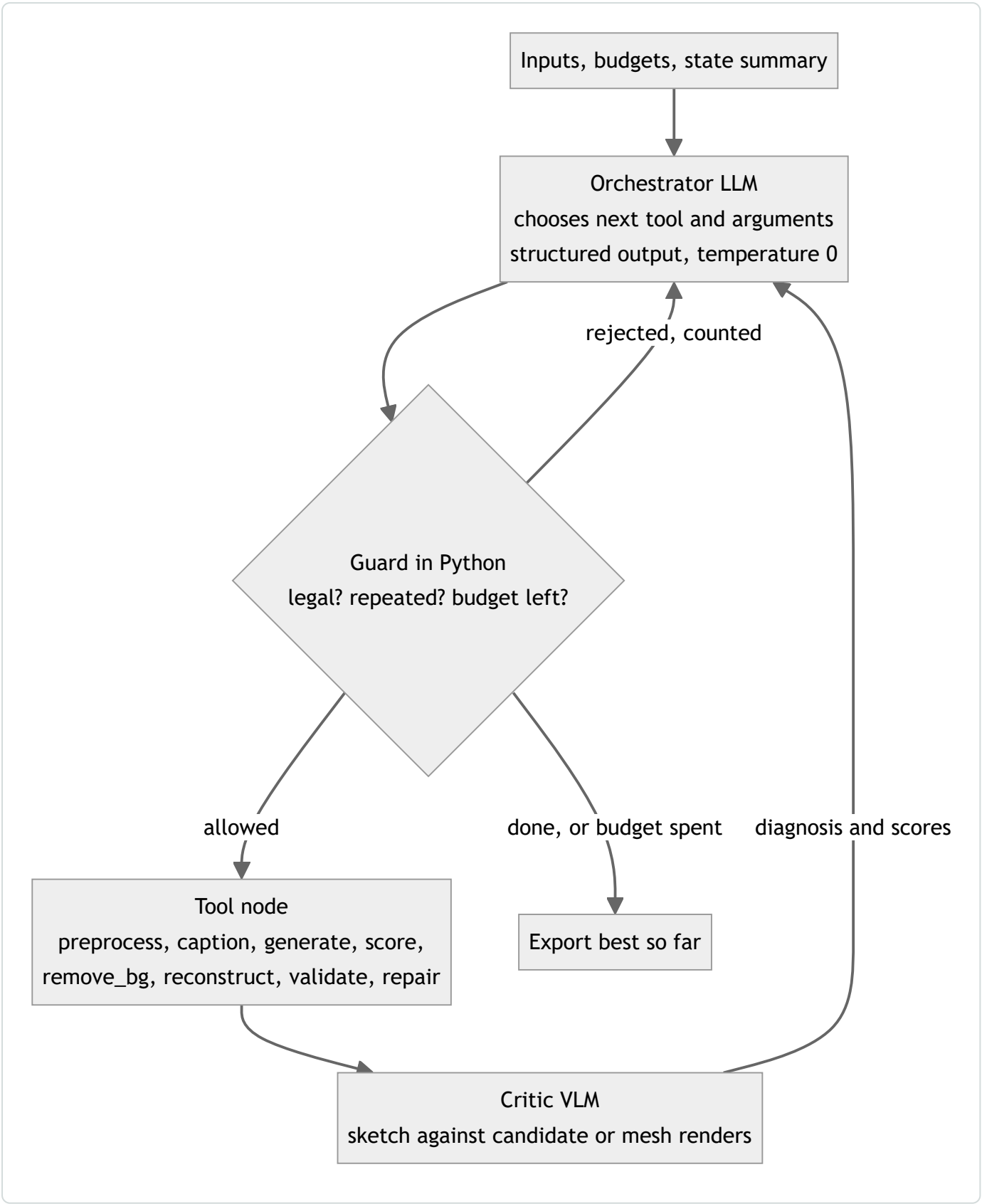
VERDICT It works and expresses the two loops neatly. The features that set LangGraph apart (durable resume of long runs, human-in-the-loop interrupts, LLM tool loops, parallel branches with reducers) are not needed here. A run takes one to two minutes, the disk cache already gives resume, and Gradio handles the human choice directly. The value in this design comes from the gates and the failure table, which Approach 1 can have equally. Moderate value, low harm if kept thin.

---

04 Approach 3: agentic LangGraph

An LLM decides which tool runs next, based on intermediate results and a critic's assessment. This is what that looks like when every component has to justify itself.

4.1 The architecture



4.2 Components and whether each earns its place

Component	Kind	Responsibility	Justified?
Orchestrator	LLM with tool calling	Picks the next tool and its arguments from a state summary	Weak The action set is small and known in advance

Component	Kind	Responsibility	Justified?
Guard	Python	Validates each action, enforces budgets, forces export at the end	**Required** It ends up doing the real control
Tools	Python	The same eight stage functions	**Same as A1**
Critic	Vision-language model	Compares sketch with candidate or mesh renders and writes a short diagnosis	**Partly** Catches semantic errors (wrong object, extra legs) that metrics miss
Captioner	Vision-language model	Names the sketched object and drafts the prompt	**Yes, as one optional call** Open vocabulary beats a fixed CLIP list

One agent or several? At most one decision-maker. Splitting it into a "generation agent" and a "reconstruction agent" would give each agent one tool and nothing to decide.

4.3 Which decisions an LLM should make

`LLM-CAPABLE`

- Naming the sketched object without a fixed vocabulary
- Writing the generation prompt
- Semantic critique of a candidate
- Choosing among remedies when given a diagnosis

`ALWAYS DETERMINISTIC`

- Every numeric threshold: fidelity scores, mesh checks
- Retry counts and the time budget
- VRAM, model loading and offload
- Termination, export format and conventions

4.4 Keeping the loop bounded

- Caps live in state: maximum decisions (for example 6), attempts per tool, and a wall-clock budget (for example 240 s on a 24 GB GPU `TEST`).
- The guard rejects actions that are illegal, repeat identical arguments, or exceed the budget, and counts rejections. Past the cap it routes to export of the best result so far. Termination belongs to the guard, never the LLM.
- `recursion_limit` stays as the last backstop; decisions use temperature 0 and a JSON schema.

4.5 What it costs

- **Latency:** each decision is a model call, roughly 1–5 s hosted or 5–15 s for a local 3–8B VLM `TEST`. A run with 4–8 calls adds about 10–60 s.
- **Resources:** either an API key plus internet on the judges' machine, or a local VLM needing roughly 6–16 GB of VRAM `TEST` on the same GPU as SDXL and Hunyuan3D-2. The team does have an LLM API key, which removes the VRAM problem, but the latency, run-to-run variation and build time remain, so the verdict below stands.
- **Reproducibility:** decisions vary between runs and API versions, which works against the 15 reproducibility points and makes failures hard to replay.
- **Build time:** tool schemas, prompts, guard, critic and testing the agent's behaviour add roughly 10–20 person-hours (section 7).

4.6 Would it improve the result?

The available remedies are few: new seeds, a different conditioning scale, a changed prompt, another background-removal method, another reconstruction seed or model. A fixed policy table applies them in a sensible order. The one new capability an agent brings is semantic critique, and a deterministic check covers most of it: require that CLIP's top label for the candidate agrees with the sketch's label, and rank by stroke recall. The expected quality gain is small and the added risk is large.

`VERDICT` Not justified for this hackathon. Keep one idea from it: the evaluator, built as deterministic scorers. The only LLM use that pays for itself is a single call that names the sketch, which the plan now includes through the team's API key.

05 Other architectures

5.1 Explicit state machine in plain Python (recommended orchestration)

Named states, a transition table mapping (state, outcome) to the next state, and a controller loop that checks budgets on every step and emits one event per transition. The events feed the UI progress panel and the manifest. This gives LangGraph's explicit routing and testability in about 150 lines with no dependency, and the table goes straight into the README.

From	Outcome	To
PREPROCESS	ok / empty	UNDERSTAND / FAILED (ask to redraw)
UNDERSTAND	any	GENERATE
GENERATE	ok / out of memory	SCORE / GENERATE at lower settings (once)
SCORE	pass / fail with rounds left / fail, none left	PREPARE / GENERATE with new seeds and scale +0.1 / PREPARE with a warning
PREPARE	ok / bad mask	RECONSTRUCT / PREPARE with the other method (once)
RECONSTRUCT	ok / error	VALIDATE / RECONSTRUCT on the next ladder rung
VALIDATE	pass / repairable / fail / ladder exhausted	EXPORT / REPAIR / RECONSTRUCT next rung / EXPORT best so far, flagged
REPAIR	any	VALIDATE (once)
EXPORT	ok	DONE
any state	budget spent	EXPORT best so far, or FAILED if nothing exists

5.2 Hybrid: deterministic pipeline with selective decisions (recommended overall)

Approach 1's stages, the state machine above, two quality gates, best-of-N generation, and a fallback ladder by GPU tier. Described fully in section 8.

5.3 UI process plus a separate GPU worker

Gradio's queue with a concurrency limit of 1 already serialises GPU jobs, which is all a single-user demo needs. Move reconstruction into a subprocess only if the spike shows its CUDA extensions crashing the whole app. Decide on evidence.

5.4 Workflow engines: Prefect, Dagster, Airflow, Temporal

Built for scheduled data jobs and durable distributed execution. Nothing here needs them. Rejected.

5.5 ComfyUI as the backend

Quick to experiment with SDXL, ControlNet and community wrapper nodes for Hunyuan3D or TRELLIS. Custom-node installs are fragile for judges to reproduce, and the result leaves little of your own code to assess under technical implementation. Rejected as a backend.

5.6 Different ML routes, which change quality more than any orchestrator

- **Sketch straight into image-to-3D (skip stage 2).** Fastest, but it runs straight into the domain gap. Keep it as a README ablation: the same sketch with and without stage 2 shows the judges why the stage exists.
- **Multi-view route** (sketch to several views, then reconstruction). InstantMesh and Unique3D belong to this family. More constraint on hidden sides, but multi-view inconsistency, heavier installs, and in one benchmark InstantMesh used about three times the

memory of most single-object models CHECKED. The PS lists them as examples and explicitly accepts "equivalent approaches", so newer 3D-native models are allowed.

- **Hosted 3D APIs.** A disclosed disaster fallback at most. They need keys and internet in the judges' environment and leave little to assess.

5.7 Model ladder by GPU

Tier	Typical GPU	Sketch to image	Image to 3D	Notes
A · 24 GB+	RTX 4090, 3090, A10G, L4	SDXL + xinsir scribble ControlNet, fp16	Hunyuan3D-2 shape + texture (16 GB together CHECKED)	Target tier. TRELLIS (16 GB, tested on Linux only CHECKED) is the alternative primary if Hunyuan's texture build fails.
B · 16 GB **Chosen**	Kaggle "GPU T4 ×2" (free); also T4, RTX 4080, 4060 Ti 16 GB	SDXL + scribble ControlNet on GPU 0, two candidates; SD 1.5 scribble if SDXL is too slow TEST	Hunyuan3D-2 on GPU 1, shape and texture run as separate steps with memory freed between them TEST	16 GB for shape plus texture is right at a T4's limit. If texturing runs out of memory, the shape plus projected colour fallback applies.
C · 8–12 GB	Laptop 3060, 4060, 4070	SD 1.5 + scribble ControlNet TEST	Hunyuan3D-2 shape only (6 GB CHECKED) with colour projected from the prepared image, or TripoSR with vertex colours TEST	Weaker texture, still meets the PS.
Avoid in 24 h	TRELLIS.2: at least 24 GB, Linux only, verified on A100/H100 CHECKED. Hunyuan3D-2.1: 10 / 21 / 29 GB for shape / texture / both CHECKED. Stable Fast 3D as primary: gated on Hugging Face under Stability's community licence CHECKED, fine as a disclosed fallback. Unique3D: no reliable requirements found and a heavy install TEST.			

06 Side-by-side comparison

Colours show how favourable each rating is for this hackathon, so "Low complexity" is green. The fourth column is the recommended hybrid from section 5.

Dimension	A1 Linear pipeline	A2 LangGraph workflow	A3 Agentic LangGraph	Hybrid (recommended)
Architectural complexity	Low Eight functions and a controller	Low–medium Same design plus state schema and reducers	High Tool schemas, guard, critic, budgets	Low–medium A1 plus a transition table
Implementation complexity	Low	Medium Serialisable state, Gradio streaming bridge	High Prompting and agent testing on top of A2	Low–medium Threshold calibration is the main extra
Development speed	Fastest End to end by about hour 6	Fast About 1–2 h behind A1	Slow Stable end to end around hours 10–14	Fastest Gates added after the first end-to-end run
Integration	Direct calls	Through state dict Must stay serialisable	Through JSON tool args LLM must produce valid arguments	Direct calls, typed contracts
Execution reliability	Predictable but blind Silent on bad outputs	Predictable	Variable Odd paths, API failures	Predictable, checks its outputs
Error handling	Basic Exceptions only unless gates are added	Good RetryPolicy plus conditional edges	Flexible but fallible Can misdiagnose; needs the guard anyway	Good Failure table plus ladder

Dimension	A1 Linear pipeline	A2 LangGraph workflow	A3 Agentic LangGraph	Hybrid (recommended)
Flexibility	Medium Edit the controller	Good Add nodes and edges	High in principle Add tools	Good Add a row and an adapter
Model orchestration	Needs a ModelManager	Same Framework does not manage VRAM	Worse Extra VLM competes for VRAM	ModelManager designed in
Debugging	Easy Plain stack traces, run folders	Medium Framework frames; tracing helps	Hard Non-deterministic decisions	Easy Manifest records every transition
Runtime overhead	Negligible	Negligible Milliseconds per step	+10–60 s per run	Negligible Best-of-N costs ~10–20 s of GPU by choice TEST
Resource consumption	Baseline	Baseline	API key, or +6–16 GB VRAM	Baseline + N candidates
Scalability	Fine for one user	Good path to LLM features	Suits future open-ended editing	Stages can be wrapped by any orchestrator later
Hackathon suitability	High	Medium–high	Low	Highest
Problem-solving value	Medium No quality control	Medium Equal to the hybrid if gates are added; the framework adds little	Low–medium Small gain, large risk	High

07 24-hour estimates

Assumptions: a team of three, free Kaggle notebooks with two T4 GPUs, and Claude Opus 5.5 writing most of the code. Figures are person-hours; the ranges mostly reflect environment risk. All are estimates TEST.

Phase	A1 Linear	Hybrid	A2 LangGraph	A3 Agentic
1 Architecture and setup: environment, CUDA extensions, weights	2–4	2–4	2–4.5	3–5.5
2 Core pipeline: stages, UI, CLI, run store	2–3	2–3	2–3	2.5–3.5
3 Model integration: SDXL + ControlNet, Hunyuan3D-2, VRAM, orientation	3–5	3–5	3–5	4–7
4 Orchestration	0.5	1.5–2.5	2.5–4	6–10
5 Testing and debugging: 25–30 sketches, thresholds, failure catalogue	4–6	5–7	5.5–8	8–12
6 Documentation and demo: README, timings, samples, clean install, rehearsal	3–4	3–4	3–4	3.5–5
Total person-hours	**14.5–22.5**	**16.5–25.5**	**18–28.5**	**27–43**
Critical path, wall clock, three people	**~13–17 h**	**~14–18 h**	**~16–20 h**	**~22–30 h**
Fits in 24 h with buffer?	Yes	Yes	Thin buffer	Only by cutting quality work

WHERE AI-ASSISTED CODING SAVES TIME

- Stage code, contracts, controller and run store
- Gradio UI and CLI
- Validation, repair and export code
- Batch evaluation script and results table
- README, Dockerfile and setup script drafts

These phases shrink by roughly half or more, and the table already assumes that.

WHERE IT SAVES ALMOST NOTHING

- Weight downloads, roughly 20–30 GB for tier A `TEST`
- Compiling Hunyuan3D-2's `custom_rasterizer` and `differentiable_renderer` against your CUDA and torch `CHECKED`
- Inference time on every test run
- Finding VRAM limits and output orientation
- Judging outputs by eye and calibrating thresholds
- A clean-machine install and demo rehearsal

Shortest path to submission

The build does not need to fill the 24 hours. The table above compares the approaches at full scope; the recommended build trims that scope to about 12–20 person-hours and a critical path of roughly 9–12 hours. The team submits at the first point where every item under "Definition of done" is true, and stops there.

DEFINITION OF DONE

- Drawing and upload both work; the prompt is optional.
- A fresh sketch becomes a textured GLB and an OBJ zip, with both gates on and a last fallback that always returns a mesh.
- Every run writes a validation report and timings.
- The README covers every item the PS lists, including measured generation time and the hardware used.
- A clean install from the README has passed the smoke test once, and the Kaggle notebook reproduces a run with Run All, without the API key.
- The repo holds at least one GLB plus example inputs and outputs.
- The demo has been rehearsed once, with a recorded backup run.

CUT OR DEFERRED

- Human pick of the candidate in the UI. One button runs everything.
- TripoSR fallback. Shape with projected colour is the last rung.
- Fast and Best presets. One setting tuned for the demo GPU, plus a documented flag for fewer candidates.
- Turntable thumbnails and the mesh silhouette check.
- A Docker image. The Kaggle notebook is the reproducible environment.

Hours	Work, in parallel where listed	Done when
0–1	Start weight downloads and the environment build. Write the stage contracts. Split roles: pipeline, UI, README.	Downloads running
1–2.5	Spike on Kaggle T4 ×2: three to five sketches through SDXL + ControlNet, then Hunyuan3D-2. Record VRAM, time and output orientation. Save the compiled extensions and weights as a private Kaggle Dataset, and check the Gradio share link.	A `.glb` exists from a sketch
2.5–5	CLI end to end with run folders. In parallel: Gradio UI against the contracts, README skeleton.	Sketch to GLB from CLI and UI; git tag `baseline`
5–7.5	Both gates, the failure table, the projected-colour fallback, export conventions, the optional API call that names the sketch.	Gates working
7.5–9.5	12–15 test sketches, some drawn by a teammate who did no tuning. Set thresholds once. The batch script writes the timings table. In parallel: README body.	Results table and sample outputs
9.5–11	Clean install from the README; finish limitations and troubleshooting; record the backup run.	Fresh install passes the smoke test
11–12	Rehearse the demo once with live drawing, then submit.	Submitted

If the environment builds cleanly this can finish around hour 9–10. If a CUDA extension fails to build, the spike shows it by hour 2.5, and the plan drops to the shape-only tier with projected colour instead of spending hours on the build.

---

08 Final recommendation

1. Which of the three would I choose, and why?

Approach 1, extended with two quality gates and a fallback ladder, written in plain Python as an explicit state machine. The control flow is fixed and short. The uncertainty is in the model outputs, and measuring them handles it better than an LLM deciding what to run. This keeps the most hours for what the rubric pays for: output quality (25), reliability and runtime (20), and reproducibility (15).

2. Is a conventional pipeline sufficient?

Yes, provided it measures its own outputs. A strictly linear version is not enough, because it exports whatever the models return. Two gates and a fallback ladder close that gap without a framework.

3. Does LangGraph give enough practical benefit?

Not for this problem. Its distinguishing features serve long, resumable, LLM-driven or human-interrupted workflows. This run lasts one to two minutes, contains no LLM, resumes from its disk cache, and gets human choice from Gradio directly. Your LangGraph experience lowers its cost but does not create a need for it. If the team uses it anyway, keep the stage functions framework-free so the graph is only wiring and can be swapped either way in about an hour `TEST`.

4. Is an agentic LangGraph system necessary?

No. It adds 10–60 s per run, non-deterministic behaviour and 10–20 person-hours, for a small action set that a policy table handles. The team's API key removes only the VRAM cost of a local model. Keep the evaluator idea and build it deterministically.

5. Is there an architecture that beats all three?

The hybrid below. It is Approach 1 with the useful parts of Approaches 2 and 3, not a new paradigm. The larger gains come from ML choices: a 3D-native reconstruction model (Hunyuan3D-2, or TRELLIS on Linux) instead of the older multi-view methods, best-of-N candidates ranked by sketch fidelity, a reconstruction-friendly prompt template, and mesh repair with consistent export conventions.

6. Would I recommend a hybrid? What does it combine?

- Deterministic stages with typed contracts (Approach 1).
- Explicit routing with bounded loops through a transition table (the useful part of Approach 2, without the framework).
- An evaluator at two gates, built from deterministic scorers (the useful part of Approach 3, without the LLM).
- Best-of-N generation, ranked automatically.
- One optional API call that names the sketch, with CLIP as the fallback so the pipeline never depends on the key.
- A model ladder by GPU tier whose last rung always returns a mesh.
- A per-run folder with manifest, and a content-hash cache.

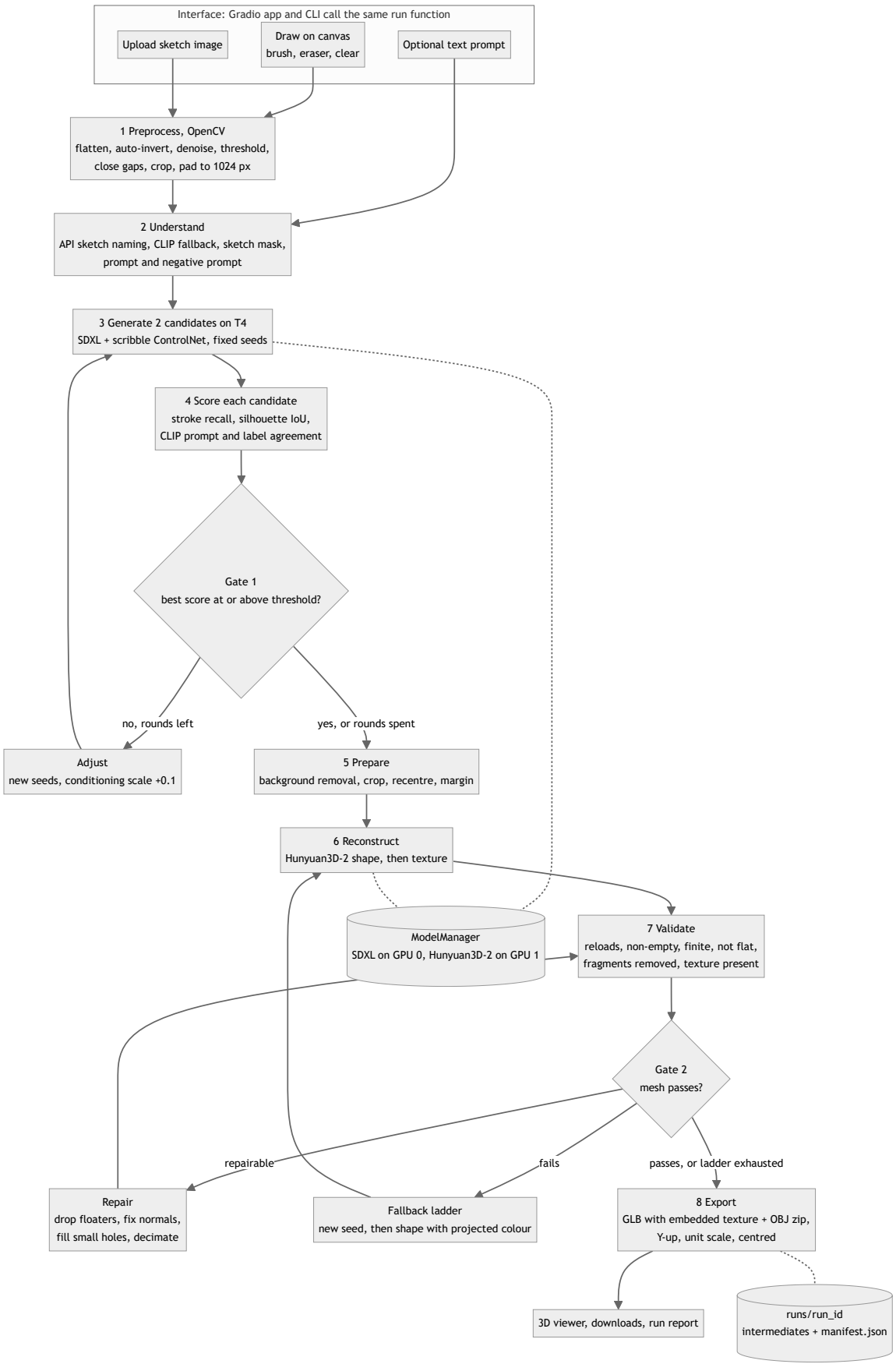
7. What does the recommended system do, from input to exported model?

1. **Input.** The user draws (brush, eraser, clear) or uploads, optionally types a prompt, and presses one button.
2. **Preprocess**, under a second: clean line art, control image, silhouette mask.
3. **Understand**: one API call names the object in a few seconds `TEST`, or CLIP does in under a second. The label is shown so a wrong guess is visible, and a typed prompt overrides it.
4. **Generate** two candidates on a T4, roughly 1–2 minutes at SDXL's 1024 px `TEST`, less with SD 1.5.
5. **Score and gate 1:** up to two regeneration rounds, otherwise the best candidate with a warning.

6. **Prepare**, 1–2 s: background removed, object centred with margin.
7. **Reconstruct**: Hunyuan3D-2 on the second T4, shape then texture as separate steps, roughly 2–4 minutes TEST.
8. **Validate and gate 2:** repair once, retry with a new seed, then step down the ladder.
9. **Export**: GLB with embedded texture plus an OBJ zip, centred, scaled, Y-up. The viewer shows the model; the run report shows scores, timings and warnings.

Target end to end: roughly 3–6 minutes on Kaggle's free T4s TEST. The README reports the measured figure and the hardware used, as the PS requires.

09 Architecture diagram



Solid arrows are the run's control flow, decided by the state machine. Dotted lines are shared services: the ModelManager serves stages 3 and 6, and every stage writes to the run folder (only export is drawn). A global time budget can send any state to Export with the best result so far.

10 Risks and decisions to settle

10.1 Decisions to settle before the event

- **GPU and OS: decided.** Free Kaggle notebooks, which run Linux on two T4s. Make sure each teammate has a phone-verified Kaggle account. Ask the organisers whether a cloud notebook may be used during the event, and whether they provide GPUs.
- **What the judges' "evaluation environment" is.** Ask the organisers: GPU or none, OS, internet access, how long they allow for setup. This decides between a Docker image, a conda script, or a backup notebook.
- **Whether pre-event setup is allowed.** If weights and environments cannot be prepared in advance, downloads start at minute zero.
- **Primary 3D model:** Hunyuan3D-2 or TRELLIS, chosen by the two-hour spike, not by reading. Start with Hunyuan3D-2.
- **External APIs.** Only the optional sketch-naming call through the team's LLM API key. Keep the key in Kaggle Secrets or an environment variable, never in the repo, and name the provider in the README.
- **Fidelity score definition**, and whether it gates or only ranks. If it does not agree with human judgement on about 20 hand-rated examples, use it for ranking only.
- **Budgets:** candidates N, regeneration rounds, reconstruction retries, global time budget per tier.
- **Output conventions:** GLB primary, OBJ zip secondary, Y-up, unit scale, centred, face budget, 1024 or 2048 px texture.
- **Team roles and contracts**, so the UI, the pipeline and the README proceed in parallel from hour 1.
- **Demo plan:** live drawing first, a recorded backup run, and a gallery of earlier outputs clearly labelled as earlier.

10.2 Risk register

Risk	Likelihood	Impact	Mitigation
CUDA extension builds fail (Hunyuan texture rasterizer, TRELLIS dependencies)	Medium on Kaggle	No texture, or no 3D at all	Kaggle runs Linux; build once in the pre-event spike and save the wheels; ladder down to shape plus projected colour
Weight downloads slow on venue Wi-Fi	Low	Hours lost	Kaggle downloads inside its own datacenter; save the weights as a Dataset once
Out of memory with both models resident	Medium	Crashes mid-run	One model family per T4; measure peaks in the spike
Domain gap: images too sketch-like to reconstruct	Medium	Flat or noisy meshes	Product-photo template, conditioning scale around 0.6–0.85 `TEST`, white background, centring
Fresh judge sketches of unexpected objects	Medium	Wrong label steers generation	Low-confidence label falls back to a generic prompt with stronger conditioning; label shown; typed prompt overrides
Control image polarity or line width wrong for the ControlNet	Medium	Ignored sketch	Check the model card and test both polarities in the spike `TEST`
Output wrongly oriented or scaled in viewers	Medium	Looks broken when judges open it	Normalise in export; open in Blender, an online glTF viewer and the in-app viewer
Texture lost on export (OBJ material paths, GLB embedding)	Medium	Grey model	Reload test inside validation; GLB as the primary format
Thin parts lost (legs, handles)	Medium	Lower fidelity	Known model limit; thicker control strokes may help `TEST`; documented in Limitations
Too slow on judges' hardware	Medium	Lost efficiency points	Documented flag for one candidate and faster shape settings
Kaggle session ends or the weekly GPU quota runs out	Medium	Lost build time	Weights and compiled extensions in a private Dataset so a restart takes minutes; spread GPU use across teammates' accounts; never run two GPU sessions on one account

Risk	Likelihood	Impact	Mitigation
Texture stage runs out of memory on a 16 GB T4	Medium	Colour-projected instead of textured model	Shape and texture as separate steps on the second T4; lower texture resolution; projected-colour fallback
Share link fails or the notebook idles out during the demo	Medium	No live demo	Test the link in the spike; keep the tab active; recorded backup run
Licence terms	Low	Disclosure gaps	Read the Hunyuan3D-2 licence for territory limits `TEST`; note Stable Fast 3D is gated; list every model in the README
Over-engineering eats hours	Medium	Weak README or tests	Timebox orchestration to 1.5 h; stop once the definition of done is met

Sources checked on 31 October 2026

- Hunyuan3D-2 repository: 6 GB shape, 16 GB shape + texture, OS support, compiled texture extensions. github.com/Tencent-Hunyuan/Hunyuan3D-2
- Hunyuan3D-2.1 README (copy on a Hugging Face Space): 10 / 21 / 29 GB, low-VRAM mode. huggingface.co/spaces/ginigen/Hunyuan3D-2.1
- TRELLIS repository: 16 GB minimum, tested on Linux, MIT, GLB export with texture. github.com/microsoft/TRELLIS
- TRELLIS.2: 24 GB minimum, Linux, A100/H100. github.com/Microsoft/TRELLIS.2, comfyui-wiki.com news
- Stable Fast 3D model card: gated, Stability Community Licence, UV-unwrapped textured mesh in under a second. huggingface.co/stabilityai/stable-fast-3d
- Single-view reconstruction benchmark including InstantMesh memory use. arxiv.org/pdf/2505.17966
- LangGraph fault tolerance (RetryPolicy) and `set_node_defaults` (timeout, cache policy, since 1.2). docs.langchain.com fault tolerance, reference.langchain.com set_node_defaults
- xinsir scribble ControlNet for SDXL (accepts varied line types and widths). huggingface.co/xinsir/controlnet-scribble-sdxl-1.0
- 2026 landscape overviews, lower weight (vendor-adjacent). cinevva.com guide, triposr.org comparison

---

Prepared for the TechZephyr 4.0 Sketch2Shape team. Problem statement: "Sketch2Shape AI: From a 2D Sketch to a 3D Asset", TechZephyr 4.0, IIT Bhubaneswar.